FUNDAMENTOS DE PROGRAMACIÓN. Curso 2024/25

SEGUNDA CONVOCATORIA. 9 de julio de 2025. Primer cuatrimestre

Se dispone de determinados datos sobre alquileres de autocares. Los datos están almacenados en el archivo `autocares.csv` cuyas cinco primeras líneas se muestran a continuación:

```
Matrícula,Marca,Plazas,Tipo,Precio,Viajes
6303KWQ,IVECO BUS,33,MINIBUS,396,[20/07/2024 23:53;24/07/2024 21:18;1879;20*28/07/2024 08:22;31/07/2024
17:46;376.5;20*06/08/2024 07:26;16/08/2024 02:09;1675.1;22]
0608GHM,MERCEDES-BENZ,26,MICROBUS,312,[18/01/2024 04:46;27/01/2024 19:19;1255.5;14*02/02/2024 17:00;06/02/2024
11:30;1297.4;11]
6738GHT,MAN,96,LUJO,1248,[07/06/2023 23:44;15/06/2023 21:17;920.1;81]
5154GBT,IRIZAR,77,LUJO,1001,[]
```

Cada línea del archivo CSV incluye la siguiente información sobre un autocar: la **matrícula**, la **marca**, el **número de plazas para viajeros**, el **tipo**, el **precio que cuesta alquilarlo diariamente** y una **lista con los viajes realizados**

Cada *Viaje* contiene la siguiente información: la **fecha y hora de inicio**, **la fecha y hora de fin**, los **kilómetros recorridos** y el **número de viajeros** transportados. Existen autocares que aún no se han alquilado para ningún viaje. Por ejemplo, se observa en el cuadro superior como el autocar de matrícula 5154GBT, no ha sido alquilado para viaje alguno.

- La primera línea es una línea de cabecera.
- En la segunda línea del archivo CSV indica que el autocar **6303KWQ**, de la marca **IVECO BUS** tiene **33** plazas para viajeros (*no se incluye las plazas destinadas al conductor y, en su caso, al guía*), es de tipo **MINIBUS**, y cuesta alquilarlo diariamente **396** euros. Ha realizado **tres viajes**: el primero se inició el **20/07/2024 a las 23:53** y finalizó el **24/07/2024 a las 21:18** realizó **1879** kilómetros y llevó a **20** viajeros. El segundo viaje lo inició el **28/07/2024 a las 08:22** y lo finalizó el **31/07/2024 a las 17:46**, habiendo realizado **376.5** kilómetros transportando a **20** viajeros. Por último, realizó un tercer viaje que se inició **el 06/08/2024 a las 07:26** y finaliza el **16/08/2024 a las 02:09**, con **1675.1** kilómetros recorridos y transportando **22** viajeros.

PROYECTO QUE SE LE REQUIERE IMPLEMENTAR

Se le facilita un proyecto Python con las siguientes carpetas:

- ***data:*** en la que se ubica el fichero *autocares.csv*
- ***src:*** que está vacía y en la que debe implementar los módulos Python ***autocares.py*** y ***autocares_test.py***

Para realizar los ejercicios que se solicitan **utilice obligatoriamente** las definiciones de los NamedTuple que se indican a continuación:

```
from typing import NamedTuple
from datetime import datetime
Viaje = NamedTuple("Viaje",
       [("inicio", datetime),   #fecha y hora de inicio del viaje.
        ("fin", datetime),      #fecha y hora de fin del viaje.
        ("kms", float),         #kilometros recorridos.
        ("num_viajeros", int)]) #número de viajeros.

Autocar = NamedTuple("Autocar",
      [("matricula", str),        #matrícula del autocar.
       ("marca", str),            #marca del autocar.
       ("plazas", int),           #número de plazas para viajeros.
       ("tipo", str),             #tipo del autocar.
       ("precio",int),            #precio que cuesta alquilarlo diariamente.
       ("viajes", list[Viaje])])  #Lista con los viajes realizados.
```

Se pide implementar las funciones de los ejercicios que figuran a continuación en el módulo ***autocares.py*** y sus correspondientes métodos de test en el módulo ***autocares_test.py*** con los mismos parámetros que se indican en los resultados esperados. Tenga en cuenta que se pueden definir funciones auxiliares cuando se considere necesario. Las puntuaciones están indicadas para cada ejercicio.

EJERCICIO 1 - (1,5 puntos + 0,25 test)

`lee_autocares`: dada una cadena de texto con la ruta de un fichero *CSV*, devuelve una lista de tuplas de tipo Autocar con la información contenida en el fichero.

```
def lee_autocares(ruta:str)->list[Autocar]:
```

Para realizar la función anterior, ayúdese de la implementación de la función

- **`parsea_viajes:`** dada una cadena de texto con los viajes separados por *, devuelva una lista de tipo Viaje. Tenga en cuenta que si la cadena de texto está vacía la función debe devolver una lista vacía (*compruébelo* en el *cuarto registro de los resultados esperados*)

  ```
  def parsea_viajes(viajes_str:str) -> list[Viaje]:
  ```

  Ayuda: Para "parsear" las fechas y horas use la cadena de formato `"%d/%m/%Y %H:%M"`

Resultados esperados:

```
Número de registros leídos: 150

Los 4 primeros registros son
Autocar(matricula='6303KWQ', marca='IVECO BUS', plazas=33, tipo='MINIBUS', precio=396,
viajes=[Viaje(inicio=datetime.datetime(2024, 7, 20, 23, 53), fin=datetime.datetime(2024, 7, 24,
21, 18), kms=1879.0, num_viajeros=20), Viaje(inicio=datetime.datetime(2024, 7, 28, 8, 22),
fin=datetime.datetime(2024, 7, 31, 17, 46), kms=376.5, num_viajeros=20),
Viaje(inicio=datetime.datetime(2024, 8, 6, 7, 26), fin=datetime.datetime(2024, 8, 16, 2, 9),
kms=1675.1, num_viajeros=22)])
Autocar(matricula='0608GHM', marca='MERCEDES-BENZ', plazas=26, tipo='MICROBUS', precio=312,
viajes=[Viaje(inicio=datetime.datetime(2024, 1, 18, 4, 46), fin=datetime.datetime(2024, 1, 27,
19, 19), kms=1255.5, num_viajeros=14), Viaje(inicio=datetime.datetime(2024, 2, 2, 17, 0),
fin=datetime.datetime(2024, 2, 6, 11, 30), kms=1297.4, num_viajeros=11)])
Autocar(matricula='6738GHT', marca='MAN', plazas=96, tipo='LUJO', precio=1248,
viajes=[Viaje(inicio=datetime.datetime(2023, 6, 7, 23, 44), fin=datetime.datetime(2023, 6, 15,
21, 17), kms=920.1, num_viajeros=81)])
Autocar(matricula='5154GBT', marca='IRIZAR', plazas=77, tipo='LUJO', precio=1001, viajes=[])
```

EJERCICIO 2 - (1,50 puntos + 0,25 el test)

`viaje_mas_kilometros`: dada una lista de tuplas *autocares* de tipo Autocar y un conjunto de marcas de autocares, devuelve una tupla formada por la matrícula y el viaje con más kilómetros recorrido entre los autocares cuya marca sea del conjunto de marcas dado. El conjunto de marcas tomará el valor por defecto None, en cuyo caso se analizarán los viajes de todos los autocares de fichero.

```
def viaje_mas_kilometros(autocares:list[Autocar], marcas:set[str]|None=None)-> tuple[str,
Viaje]:
```

Resultados esperados:

```
El viaje con más kilómetros es: ('4537HHL', Viaje(inicio=datetime.datetime(2023, 12, 12, 9, 3), fin=datetime.datetime(2023, 12, 14, 2, 10), kms=1999.6, num_viajeros=108))

El viaje con más kilómetros entre las marcas {'IRIZAR', 'SETRA', 'MERCEDES-BENZ'} es: ('0768LNG', Viaje(inicio=datetime.datetime(2024, 11, 25, 0, 27), fin=datetime.datetime(2024, 12, 4, 5, 21), kms=1985.1, num_viajeros=78))
```

EJERCICIO 3 - (1,75 puntos + 0,25 el test)

autocar_mayor_beneficio: dada una lista de tuplas *autocares* de tipo **Autocar,** devuelve la matrícula, la marca y el beneficio del autocar con mayor beneficio.

El beneficio es el resultado de multiplicar el precio diario por el número de días que ha estado alquilado un autocar dividido entre los kilómetros recorridos. A estos efectos, la propiedad days devuelve los días de diferencia entre dos fechas f1 y f2: (f1-f2).days. *En nuestro caso, tenga en cuenta que, si un autocar inicia el viaje en la fecha 20/10/2024 y lo finaliza el 22/10/2024, la diferencia devolverá 22-20=2, cuando se deben considerar tres días (20, 21 y 22).*

```
def autocar_mayor_beneficio(autocar:list[Autocar])-> tuple[str,str,float]:
```

Resultados esperados:

```
El autocar con mayor beneficio es: ('4955LBB', 'MAN', 135.33573490510244)
```

EJERCICIO 4 - (1,75 puntos + 0,25 el test)

promedio_viajeros_por_tipo: dada una lista de tuplas de tipo Autocar y una fecha, que por defecto tomará el valor None, debe devolver un diccionario que a cada tipo de autocar le asocie el promedio de viajeros que estaban de viaje en la fecha dada. Si la fecha toma el valor None, se tendrán en cuenta todos los viajes.

Nota: observe que en los resultados esperados se visualizan todos los tipos de autobús y en el caso de que no haya habido viajeros en la fecha indicada para algún tipo (en el test se usa 10 de febrero de 2024), el resultado es *None* para el tipo de que se trate.

```
def promedio_viajeros_por_tipo(autocares: list[Autocar], fecha:date|None=None)
      ->dict[str,float|None]:
```

Resultados esperados

```
El promedio de viajero por tipo es: {'MINIBUS': 24.6, 'MICROBUS': 16.86842105263158, 'LUJO': 78.80232558139535, 'DOBLE PISO': 101.21698113207547, 'ESTÁNDAR': 51.22727272727273}

El promedio de viajero por tipo que estaban de viaje el 2024-02-10 es: {'MINIBUS': None, 'MICROBUS': 23.0, 'LUJO': 88.5, 'DOBLE PISO': 98.0, 'ESTÁNDAR': None}
```

EJERCICIO 5 - (2,25 puntos + 0,25 el test)

`meses_mas_viajeros_por_tipo:` dada una lista de tuplas de tipo **Autocar** y un número entero n, debe devolver un diccionario que a cada tipo de autocar le haga corresponder una lista de tuplas con los n meses en que han transportado más viajeros. Para cada viaje se tomará el mes de la fecha de inicio y no se tendrá en cuenta el año. Las listas de tuplas estarán formadas por el número del mes y la suma de viajeros trasportados en ese mes y estarán ordenadas de mayor a menor suma de viajeros.

```
def meses_mas_viajeros_por_tipo(autocares:list[Autocar],n:int)->dict[str,list[tuple[int,int]]]:

```

Resultados esperados

```
Los 4 meses con más viajeros por tipo son
('MINIBUS', [(9, 160), (12, 147), (7, 119), (4, 108)])
('MICROBUS', [(1, 117), (11, 105), (2, 80), (6, 79)])
('LUJO', [(11, 1021), (5, 986), (7, 832), (4, 691)])
('DOBLE PISO', [(12, 1668), (9, 1483), (3, 1135), (8, 1054)])
('ESTÁNDAR', [(6, 1010), (12, 682), (4, 585), (11, 585)])
```